Milestone 1 – Task 2: System Design Document

AI Code Review & Security Analysis Agent · Infosys Springboard Internship · Week 1–2

1. Overview

1.1 Goal

A developer pastes code or uploads a Python/Java file. The platform validates it, runs a multi-agent pipeline (quality review, security scan, remediation, summary), shows severity-scored findings in a portal, lets the developer chat with a RAG-grounded assistant about any finding, and exports a review report.

1.2 Scope

In scope	Out of scope (for now)
Python and Java single-file analysis	Whole-repository or multi-file dependency analysis
OWASP-category vulnerability detection	Runtime / dynamic testing
Fix suggestions with corrected code	Auto-committing fixes to Git
RAG chat over a secure coding knowledge base	Languages other than Python/Java
PDF/JSON/Markdown report export	User accounts with roles (optional later)

1.3 Assumptions

- Files are limited to **200 KB / 2,000 lines** per submission.
- One LLM provider is used behind an interface so it can be swapped.
- A free local embedding model is used for RAG.
- Findings are **advisory**; the tool never executes submitted code.

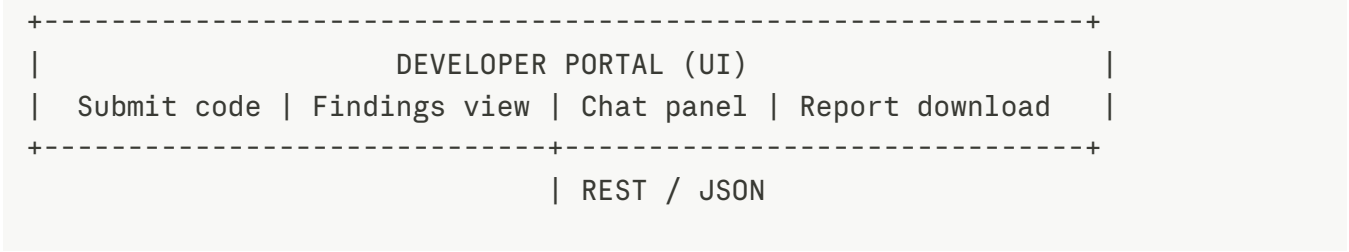
---

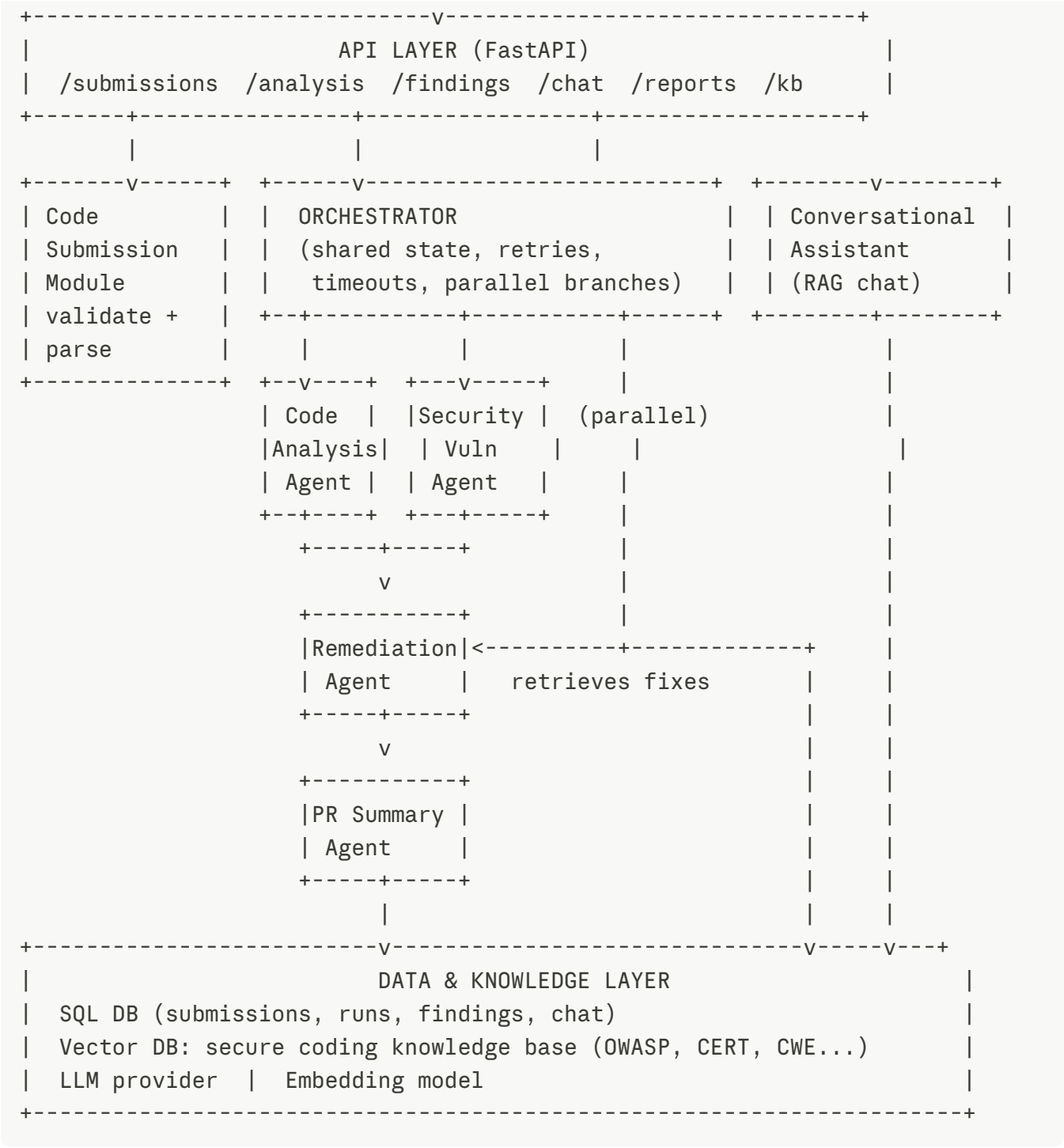
2. Recommended Technology Stack

Layer	Choice	Reason
Language	Python 3.11+	Best AI/RAG ecosystem
Backend API	FastAPI	Async, automatic OpenAPI docs, Pydantic validation
Frontend	Streamlit (fast) or React (richer)	Streamlit lets you finish the portal in days
Orchestration	LangGraph, or a custom async orchestrator	Explicit state graph, parallel branches
LLM	Claude / Gemini / OpenAI / Groq via a `LLMClient` wrapper	Swappable, structured JSON output
Embeddings	`sentence-transformers` (`all-MiniLM-L6-v2` or `bge-small`)	Free, local
Vector store	ChromaDB (persistent)	Metadata filtering, easy
Relational DB	SQLite (dev) → PostgreSQL (later) via SQLAlchemy	Stores runs, findings, chat
Code parsing	`ast` + `radon` (Python), `javalang` or `tree-sitter` (Java)	Syntax validation and metrics
Static helpers	Bandit, Semgrep rules (optional)	Boost security precision
Reports	Jinja2 + WeasyPrint/ReportLab; JSON; Markdown	Exportable review reports
Testing	pytest	Unit and integration tests

---

3. High-Level Architecture





3.1 Layer responsibilities

Layer	Responsibility
Presentation	Collect code, show findings, chat, download reports. No business logic.

Layer	Responsibility
API	Validate requests, start analysis runs, return results. Thin.
Orchestration	Run agents in the right order, share state, handle failure.
Agents	One focused job each; read and write the shared state.
Knowledge	Ingest, chunk, embed, store, and retrieve secure coding content.
Persistence	Durable storage of everything the user may revisit.

---

4. Module Breakdown (maps to the six project modules)

#	Module	Key components	Milestone
1	Code Submission	Paste box, file upload, language detection, syntax validation, size limits	M1
2	Knowledge Base & RAG	Loader, chunker, embedder, Chroma index, retriever	M1
3	Orchestration & Analysis Pipeline	Orchestrator, 4 agents, shared state	Later
4	Findings & Severity Scoring	Scoring engine, findings table, filters	Later
5	Conversational Assistant	Chat service, session memory, retrieval + generation	Later
6	Report Generation & Export	Templates, PDF/JSON/MD export	Later

---

5. Agent Responsibilities

5.1 Summary table

Agent	Purpose	Input	Output	Uses RAG?
Code Analysis Agent	Quality, smells, design, complexity	Code, AST, metrics	List of `Finding` (type = quality)	Light (optional)
Security Vulnerability Agent	OWASP vulnerabilities	Code, AST, static-rule hits	List of `Finding` (type = security)	Optional (CWE confirmation)
Remediation Agent	Fix per finding	Finding + code snippet + retrieved guidance	`Remediation` per finding	**Yes**
PR Summary Agent	Human-readable review	All findings and remediations	`ReviewSummary`	No
Conversational Code Assistant	Follow-up Q&A	Question + finding context + history	Answer with citations	**Yes**

5.2 Code Analysis Agent

- **Deterministic step:** compute cyclomatic complexity, function length, nesting depth, parameter count, class size, duplication. Detect unused imports/variables, broad exception handling, magic numbers.
- **LLM step:** receives code plus the computed metrics and flags design issues rules cannot see (God class, feature envy, poor naming, SRP violations).
- **Output rules:** every finding has line range, smell name, short explanation, and a quality severity (info/low/medium/high). Must not report security issues (that is the other agent's job).

5.3 Security Vulnerability Agent

- **Deterministic step:** rule/pattern scan (regex + AST) for known sinks: string-built SQL, `os.system`, `eval`, `pickle.loads`, `yaml.load`, `MD5`, hardcoded secret patterns (high-entropy strings, variables named `password`, `api_key`, `token`).
- **LLM step:** confirms or rejects each candidate (reduces false positives), traces whether user input reaches the sink, and finds issues rules miss (broken access control, CSRF, insecure auth flow).
- **Output rules:** every finding carries OWASP category, CWE ID, line range, `confidence`, exploit explanation, and severity. Must return strict JSON.

5.4 Remediation Agent

- For each finding it builds a retrieval query (e.g., "parameterised queries Python sqlite3"), fetches top-k chunks filtered by language/category, then asks the LLM for: explanation of the fix, **corrected code**, and cited sources.
- Must keep the original behaviour of the code and change only what is needed.
- Batches similar findings to save LLM calls; falls back to a template fix from the knowledge base if the LLM fails.

5.5 PR Summary Agent

- Aggregates findings into a pull-request-style review: overall verdict (Approve / Needs changes / Block), counts by severity, top 3 risks, quality highlights, remediation roadmap ordered by priority (fix now / next sprint / nice to have).
- Does **not** invent findings; works only from structured state.

5.6 Conversational Code Assistant

- Session is tied to an `AnalysisRun`, so it knows the code, findings and fixes.
- Pipeline per message: classify intent (explain finding / how to fix / general guideline) → retrieve chunks → prompt with context and last N turns → answer with citations → save message.
- If retrieval confidence is low it answers "not found in the knowledge base" rather than guessing.

5.7 Common agent contract

Every agent is a class implementing the same interface, so it can be tested and swapped independently:

```
class BaseAgent(Protocol):
    name: str
    async def run(self, state: PipelineState) -> PipelineState: ...
```

Prompts live in separate template files (`prompts/security_agent.md`) and each agent must return **schema-validated JSON** (Pydantic). Invalid JSON triggers one repair retry.

---

6. Orchestration Flow

6.1 Sequence

```
1. Submit code  -->  validate (syntax, size, language)  -->  create
Submission
2. Create AnalysisRun (status = QUEUED) and PipelineState
3. Preprocess: parse AST, compute metrics, run static rule scan
4. FAN-OUT (parallel):
      A) Code Analysis Agent    B) Security Vulnerability Agent
5. FAN-IN: merge findings, deduplicate, assign severity score
6. Remediation Agent (per finding, RAG-grounded)
7. PR Summary Agent
8. Save results, status = COMPLETED, return to UI
9. Any time after step 8: Conversational Assistant (RAG chat)
10. Export report on demand
```

6.2 Why this order

- Steps 4A and 4B are independent, so **parallel execution** cuts total time.
- Remediation needs the **final, deduplicated** list of findings.
- Summary needs findings **and** fixes.
- Chat is separate from the pipeline so it can be used repeatedly.

6.3 Run status lifecycle

`QUEUED → PARSING → ANALYZING → REMEDIATING → SUMMARIZING → COMPLETED`, with `FAILED` and `PARTIAL` as terminal states. `PARTIAL` means at least one agent failed but others succeeded, and the UI still shows what it has.

6.4 Error handling rules

Situation	Behaviour
Syntax error in code	Reject at submission with line/column message; no pipeline run
LLM timeout / rate limit	Retry up to 2 times with exponential backoff
Invalid JSON from LLM	One repair prompt, then mark agent step failed
One agent fails	Continue; run becomes PARTIAL; show warning
Retrieval returns nothing	Remediation uses generic template; chat says not found
Very large file	Split by function/class and analyse chunks, then merge

6.5 Shared state object

```
class PipelineState(BaseModel):
    run_id: str
    language: Literal["python", "java"]
    source_code: str
    ast_summary: dict | None = None
    metrics: dict | None = None
    static_hits: list[dict] = []
    quality_findings: list[Finding] = []
    security_findings: list[Finding] = []
    merged_findings: list[Finding] = []
    remediations: list[Remediation] = []
    summary: ReviewSummary | None = None
    errors: list[AgentError] = []
    status: RunStatus = "QUEUED"
```

---

7. Data Models

7.1 Entity relationships

```
Submission 1 ---- * AnalysisRun 1 ---- * Finding 1 ---- 0..1 Remediation
                         |
AnalysisRun 1 ---- 0..1 ReviewSummary
AnalysisRun 1 ---- 0..1 Report
AnalysisRun 1 ---- * ChatSession 1 ---- * ChatMessage
KnowledgeDocument 1 ---- * KnowledgeChunk   (vector store)
```

7.2 Relational tables

Submission

Field	Type	Notes
id	UUID, PK	
source_type	enum(`paste` , `upload`)	
filename	str, null	For uploads
language	enum(`python` , `java`)	Detected + confirmed
code	text	Raw source
code_hash	str	SHA-256, to detect duplicates
line_count, size_bytes	int	
syntax_valid	bool	
syntax_errors	JSON	List of `{line, col, message}`
created_at	datetime	

AnalysisRun

Field	Type	Notes
id	UUID, PK	
submission_id	FK	

Field	Type	Notes
status	enum	See lifecycle
started_at, finished_at	datetime	
metrics	JSON	Complexity etc.
overall_score	int 0–100	Quality/security health
llm_model	str	For reproducibility
error_log	JSON	Failed steps

Finding

Field	Type	Notes
id	UUID, PK	
run_id	FK	
category	enum(`security` , `quality`)	
title	str	e.g., "SQL Injection"
owasp_category	str, null	e.g., "A03:2021 Injection"
cwe_id	str, null	e.g., "CWE-89"
smell_type	str, null	e.g., "Long Method"
severity	enum(`critical` , `high` , `medium` , `low` , `info`)	
severity_score	float 0–10	Computed
confidence	float 0–1	Agent certainty
line_start, line_end	int	
code_snippet	text	Affected lines
description	text	What and why
source_agent	str	

Field	Type	Notes
status	enum(`open` , `fixed` , `ignored` , `false_positive`)	

Remediation

Field	Type	Notes
id	UUID, PK	
finding_id	FK	
explanation	text	Why this fix works
fixed_code	text	Corrected code
steps	JSON	Ordered fix steps
references	JSON	`[{title, url, chunk_id}]`
effort	enum(`low` , `medium` , `high`)	

ReviewSummary: `run_id` , `verdict` , `severity_counts` (JSON), `top_risks` (JSON), `narrative` (text), `roadmap` (JSON).

Report: `run_id` , `format` (pdf/json/md), `file_path` , `generated_at` .

ChatSession: `id` , `run_id` , `created_at` . **ChatMessage:** `id` , `session_id` , `role` (user/assistant), `content` , `finding_id` (nullable), `citations` (JSON), `created_at` .

7.3 Knowledge base (vector store) schema

Field	Notes
chunk_id	Stable ID (`doc_id#n`)
text	Chunk content (with heading path prefix)
embedding	Vector (384 dims for MiniLM)
source_doc	e.g., "OWASP SQL Injection Prevention Cheat Sheet"
source_url	For citation

Field	Notes
doc_type	`cheatsheet`, `top10`, `asvs`, `cert`, `cwe`, `custom`
owasp_category	e.g., A03
cwe_ids	list
language	`python`, `java`, `general`
section_path	Heading hierarchy
has_code	bool
ingested_at	datetime

7.4 Pydantic models (core, for agent I/O)

```
class Severity(str, Enum):
    CRITICAL = "critical"; HIGH = "high"; MEDIUM = "medium"
    LOW = "low"; INFO = "info"

class Finding(BaseModel):
    id: str
    category: Literal["security", "quality"]
    title: str
    owasp_category: str | None = None
    cwe_id: str | None = None
    smell_type: str | None = None
    severity: Severity
    severity_score: float
    confidence: float = Field(ge=0, le=1)
    line_start: int
    line_end: int
    code_snippet: str
    description: str
    source_agent: str

class Remediation(BaseModel):
    finding_id: str
    explanation: str
```

```
    fixed_code: str
    steps: list[str]
    references: list[Reference]
    effort: Literal["low", "medium", "high"]
```

7.5 Severity scoring design

A simplified, explainable CVSS-inspired formula:

```
severity_score = round( base_impact x exploitability x confidence_factor , 1
)   (capped at 10)
```

Vulnerability class	Base impact (0–10)
RCE / command injection, SQL injection, deserialization	9.0–9.8
Hardcoded secrets, broken access control	7.5–8.5
XSS, CSRF, path traversal	6.0–7.5
Weak crypto, insecure config	4.0–6.0
Quality smells	1.0–5.0 (by complexity / size)

- **exploitability** 0.7–1.0: higher if user input demonstrably reaches the sink.
- **confidence_factor** = agent confidence (0.5–1.0).
- **Mapping:** 9.0–10 Critical, 7.0–8.9 High, 4.0–6.9 Medium, 0.1–3.9 Low, 0 Info.
- **Overall health score:** `100 − min(100, Σ weighted penalties)`, with weights Critical 25, High 12, Medium 5, Low 2.

The scoring engine is **deterministic code, not the LLM**, so results are reproducible and explainable to a mentor.

7.6 Sample finding (JSON)

```
{
  "id": "f-0001",
  "category": "security",
  "title": "SQL Injection",
  "owasp_category": "A03:2021 Injection",
  "cwe_id": "CWE-89",
  "severity": "critical",
  "severity_score": 9.4,
  "confidence": 0.95,
  "line_start": 14,
  "line_end": 14,
  "code_snippet": "cur.execute(\"SELECT * FROM users WHERE id=\" + uid)",
  "description": "User-controlled 'uid' is concatenated into a SQL string.",
  "source_agent": "security_agent"
}
```

---

8. Key Component Designs

8.1 Code Submission Module (Milestone 1 build)

```
Input (paste/upload) -> size & type check -> language detect -> syntax
validate
   -> if invalid: return errors (line, col, message)
   -> if valid: store Submission, return submission_id
```

Check	Python	Java
Extension	`.py`	`.java`
Syntax validation	`ast.parse(code)` → catches `SyntaxError` with line/col	`javalang.parse.parse(code)` or tree-sitter error nodes
Language detection (paste)	User picks, with heuristic fallback (`def` / `import` vs `public class` / `;`)	Same

Safety: never `exec` / `eval` submitted code; read uploads as UTF-8 text only; reject binaries; enforce size limits; sanitise filenames.

8.2 Knowledge Base Ingestion Pipeline (Milestone 1 build)

```
sources/ (md, html, pdf, txt)
   -> loader (clean, strip nav/boilerplate)
   -> structure-aware chunker (400-800 tokens, 10-15% overlap, keep code
blocks)
   -> metadata tagging (owasp, cwe, language, source_url)
   -> embedder (sentence-transformers)
   -> Chroma collection "secure_coding_kb"
   -> smoke test: 10 sample queries, print top-3 with scores
```

Re-indexing is idempotent: chunk IDs are stable hashes, so re-running updates rather than duplicates.

8.3 Retrieval design

- Query = finding title + CWE + language + short code context.
- `top_k = 5`, metadata filter on `language in (lang, general)`.
- MMR to avoid duplicate chunks; discard chunks below a similarity threshold.
- Prompt format: numbered context blocks `[1]`, `[2]` so the answer can cite `[1]`.

---

9. API Design (REST)

Method	Endpoint	Purpose
POST	`/api/submissions`	Paste or upload code; validates and stores
POST	`/api/analysis`	Start pipeline for a submission; returns `run_id`
GET	`/api/analysis/{run_id}`	Status and progress
GET	`/api/analysis/{run_id}/findings`	Findings with filters (severity, category)
GET	`/api/findings/{id}`	Single finding with remediation
POST	`/api/chat`	Ask a question (`run_id`, `finding_id?`, `message`)

Method	Endpoint	Purpose
GET	`/api/reports/{run_id}?format=pdf|json|md`	Export report
POST	`/api/kb/ingest`	(Admin) Index documents
POST	`/api/kb/search`	(Debug) Test retrieval
GET	`/api/health`	Liveness check

---

10. Portal (UI) Layout

Screen	Contents
Submit	Language selector, code editor box / file upload, "Validate & Analyse" button, syntax error display
Results	Health score, severity counts, filterable findings table, code viewer with highlighted lines
Finding detail	Description, OWASP/CWE tags, vulnerable code vs fixed code diff, references
Chat panel	Persistent side panel, finding-aware suggestions, citations shown
Report	Preview and download buttons (PDF/JSON/Markdown)

---

11. Non-Functional Design

Concern	Approach
Security of the tool itself	Never execute user code; secrets in `.env`; input size limits; rate limit API; sanitise rendered output (avoid XSS in our own UI)
Privacy	Warn that code is sent to an LLM provider; do not log full code in plain logs

Concern	Approach
Performance	Parallel agents; cache by `code_hash`; chunk big files; target < 60 s for 500 lines
Reliability	Retries, partial results, status tracking
Explainability	Every finding carries evidence, CWE, confidence and citations
Observability	Structured logging with `run_id`; store LLM model and prompt version per run
Testability	Agents behind interfaces; mock LLM in tests; golden test files with known vulnerabilities
Maintainability	Prompts in files; config in YAML/env; modular folders

---

12. Proposed Project Structure

```
code-review-agent/
├── app/
|   ├── main.py                # FastAPI entry
|   ├── api/                   # routers: submissions, analysis, chat,
reports, kb
|   ├── core/                  # config, logging, LLM client, security utils
|   ├── models/                # SQLAlchemy models + Pydantic schemas
|   ├── submission/            # validators (python_validator.py,
java_validator.py)
|   ├── analysis/              # ast_utils.py, metrics.py, static_rules.py
|   ├── agents/                # base.py, code_analysis.py, security.py,
remediation.py, pr_summary.py, assistant.py
|   ├── orchestrator/          # pipeline.py, state.py
|   ├── rag/                   # loader.py, chunker.py, embedder.py,
store.py, retriever.py
|   ├── scoring/               # severity.py
|   └── reports/               # templates/, generator.py
├── prompts/                   # one .md per agent
├── knowledge_base/            # raw OWASP/CERT/CWE source documents
├── ui/                        # Streamlit or React app
```

```
├── tests/                          # unit, integration, golden samples
├── docs/                           # research + design documents
├── .env.example
└── requirements.txt
```

---

13. Risks and Mitigations

Risk	Mitigation
LLM hallucinated vulnerabilities	Static-rule evidence + confidence + confirm step + `false_positive` status
Inconsistent JSON output	Pydantic validation, repair retry, low temperature
Poor retrieval quality	Structure-aware chunks, metadata filters, test query set
API cost / rate limits	Cache by hash, batch findings, free embeddings, option for small model
Large files exceed context	Function-level chunking and merge
Scope creep	Freeze scope to Python/Java single files for this internship